INTERNAL ENGINEERING / CASE REVIEW

The Holder Double-Vote Concern

APELEVEL

Investigation, evidence, first-ballot enforcement, and the new wallet-aware voting interface.

PARLAY GOVERNANCE / Robinhood Chain

Prepared by **ApeLevel Gaming & Branding** for PARLAY

23 October 2026 • Internal development record

Outcome: One stored ballot was confirmed for the holder’s wallet and Proposal #1. Database uniqueness and the vote-insertion method protect the first successfully recorded ballot. A new interface check was developed so connected wallets with recorded ballots see a disabled, explanatory vote button.

Live platform: https://parlay.apelevel.workers.dev/

Deployment reference (reported successful): e5a16d22-30a5-47e8-9cf6-febf881ead95

01 | The holder’s concern

A PARLAY holder tested the governance system from an iPhone using MetaMask and a separate wallet holding a small amount of $PARLAY. The holder reported that the wallet connection and voting flow worked notwithstanding a wallet warning message. The holder then described choosing **“Yup.”**, reconnecting, and attempting another vote for **“Yes.”** on the same proposal.

From the holder’s perspective, the interface appeared to permit a second voting attempt. The practical concern was whether a wallet could cast duplicate ballots, improperly increase its voting influence, or change its earlier selection. The holder also suggested that the interface should explicitly indicate when a wallet had already voted after reconnecting.

ApeLevel’s immediate response

ApeLevel responded that the voting engine measures each wallet’s token holdings at the same closing blockchain snapshot, so the wallet’s token balance cannot legitimately be counted twice merely because the participant presses the button more than once. ApeLevel said it would examine how the next voting window finalized and continue improving the voting experience.

Technical distinction: A single balance snapshot alone does not prevent duplicate ballot records. Ballot uniqueness and first-vote enforcement are separate database/application safeguards. ApeLevel therefore investigated the recorded ballot and the underlying submission logic rather than relying solely on its initial explanation.

02 | Evidence collected

ApeLevel queried the production Cloudflare D1 database using read-only SQL, inspecting the specific holder wallet. The following findings were directly observed in the returned records:

Check	Confirmed database result
Wallet / proposal	0x715f15c47fbe562b2920274f2a81db6879f1d65a / Proposal #1
Ballot count	1 recorded ballot
Recorded ballot ID	5
Stored option ID	1
Proposal option mapping	1 = “Yes.” / 2 = “Yup.”

The holder's wallet therefore had **one** ballot for Proposal #1, and its stored selection was **"Yes."** The investigation found no duplicate record for that wallet and proposal. The database rows alone cannot establish whether the initial "Yup." attempt failed, was never submitted, or was otherwise unsuccessful; they establish only the final stored record.

03 | The original ballot already had protection

Review of the live database schema showed a case-insensitive wallet-address field and this uniqueness rule:

Database safeguard	Effect
UNIQUE(proposal_id, wallet_address)	No two rows can exist for the same wallet on the same proposal; different proposals remain independent.
COLLATE NOCASE on wallet_address	Letter-case differences in a wallet address do not create a second identity.

Inspection of the Worker's **POST /api/votes** code found that the submission uses **INSERT OR IGNORE INTO votes**. If that insertion does not create exactly one row, the backend returns HTTP **409** with a ballot-not-accepted response. The endpoint does not overwrite the existing option or signature with an update or replacement operation.

Accordingly, the **first successfully inserted ballot prevails**, even if a wallet attempts to submit a later ballot choosing another option. A separate snapshot-finalization operation updates the voting_power field only; it does not change the stored selection.

04 | Improvement introduced to remove confusion

Although the database behaved as intended, ApeLevel accepted that the user interface could better communicate the protection to returning holders. The frontend had previously disabled its vote button after a successful submission within the current page session, but a reconnect or refresh could lose that local confirmation.

ApeLevel added a read-only **GET /api/votes/status** endpoint. It looks up the connected wallet and returns the IDs of proposals for which the wallet already has a recorded ballot, without exposing the chosen option. The React frontend requests this status when a wallet connects and checks the status separately against each proposal's ID.

Connected-wallet state	Voting-button behaviour
Existing ballot for this proposal	Disabled: “THIS WALLET HAS ALREADY CAST A BALLOT”
No ballot for another open proposal	Voting remains available after selecting an option.
Lookup pending	Disabled: “CHECKING BALLOT STATUS…”
Lookup fails / unverified	Disabled until the status can be confirmed (fail-closed).

This is a **usability and defense-in-depth enhancement**, not a change to the underlying voting rights. A wallet that voted on Proposal #1 is not blocked from voting on a different open proposal. When a status query fails, the frontend temporarily holds voting disabled to avoid a misleading or unverified state; the backend uniqueness constraint remains the authoritative guard.

05 | Testing, deployment, and verification

The revised Worker and frontend source passed **npm run build** locally. The changes were committed to the PARLAY GitHub repository and pushed to the **main** branch after resolving an out-of-date local branch with a successful rebase.

The first live API check returned **“API endpoint not found”**. Subsequent Cloudflare deployment logs revealed the cause: its selected build token had been deleted or rotated, preventing several recent GitHub builds from deploying. This was a **deployment-authentication issue**, not a TypeScript compilation failure. ApeLevel created and saved a new Cloudflare build token, then retried the build.

The deployment was subsequently **reported successful**, with the following Cloudflare deployment identifier:

Deployment field	Recorded value
Platform	Cloudflare Workers — PARLAY
Deployment ID	e5a16d22-30a5-47e8-9cf6-febf881ead95
GitHub commit pushed	de76f58
Date	23 October 2026

The deployment ID provides an internal reference to the specific Cloudflare release. It is not an authentication secret or independent proof of correctness. The user also reported that the application

worked after deployment; an independently documented browser reconnection test for the holder’s wallet was not provided in this review.

06 | Findings and disposition

Confirmed: The database held only one ballot for the holder on Proposal #1. Its stored choice was “Yes.” The schema prevented duplicate rows per wallet and proposal, and the voting endpoint used first-insert-wins semantics rather than overwriting the original ballot.

Resolved as a product issue: ApeLevel implemented a wallet-specific status lookup and disabled-button message so a returning wallet is notified it already cast a ballot on that particular proposal. This reduces the possibility that the interface could imply a second vote had been counted.

Not established: Available records do not prove whether the holder’s initial “Yup.” selection reached the server. They also do not independently establish completion of a formal end-to-end regression test for the newly deployed interface on every wallet/browser combination.

The overall conclusion is that **the original one-ballot rule operated as intended in the inspected records**; ApeLevel nevertheless strengthened the user experience to communicate that rule clearly and preserve a predictable, proposal-specific ballot lockout after reconnecting.

Governance website: parlay.apelevel.workers.dev
Repository: github.com/apelevel/parlay